

Regression

Lesson 3 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

9 October 2026 · reading time about 75 minutes

Contents

Lesson plan	2
1. Why regression comes first	2
2. The model and the cost	3
2.1 What a linear model claims	3
2.2 Why squared error	3
2.3 A worked example	4
3. The exact solution	4
3.1 The normal equation, derived	4
3.2 A worked example, by hand	5
3.3 When the exact solution is not available	6
4. Gradient descent	6
4.1 The update rule, derived	6
4.2 A worked step	7
4.3 Choosing the learning rate	8
4.4 How stretched the valley is	8
5. Curves, and the price of flexibility	9
5.1 A linear model that bends	9
5.2 What it costs	9
5.3 The mechanism	9
6. Regularisation	10
6.1 Two penalties	10
6.2 Ridge has a closed form	10
6.3 Why Lasso reaches zero and Ridge does not	11
6.4 Worked: what a penalty buys	12
6.5 Worked: Lasso as a feature selector	12
7. Reading coefficients	13
7.1 What a coefficient means, exactly	13
7.2 Worked: when the estimate can be trusted	13
7.3 Worked: when it cannot be trusted at all	13
7.4 The warning that belongs on every coefficient	14
8. Before the next lesson	14
Further reading	14

Lesson plan

Time	Minutes	Segment	Material
0:00–0:08	8	Exercise 2 returned; the dataset	Slides 2–5
0:08–0:23	15	The model and its cost function	Slides 6–12
0:23–0:37	14	The exact solution, and when it fails	Slides 13–18
0:37–0:48	11	Gradient descent	Slides 19–23
0:48–1:13	25	Notebook 01 — regression from scratch	Slide 24
1:13–1:28	15	Break	Slide 25
1:28–1:45	17	Curves, and the price of flexibility	Slides 26–33
1:45–2:05	20	Notebook 02 — polynomials and overfitting	Slide 34
2:05–2:26	21	Ridge and Lasso	Slides 35–44
2:26–2:35	9	Reading coefficients honestly	Slides 45–48
2:35–2:55	20	Notebook 03 — ridge, lasso, collinearity	Slide 49
2:55–3:00	5	Summary; homework set	Slides 50–51
	180	Total	51 slides, 3 notebooks

Slide 1 is the title slide, so the numbers above match the page numbers in `Slides/regression_slides.pdf`. The lecture segments come to 100 minutes across 46 content slides — a shade under 28 slides per hour.

1. Why regression comes first

This is the first method in the course, and it is a good first method for three reasons that have nothing to do with it being simple.

It has an **exact solution**, so we can see what “fitting a model” means without any iterative machinery in the way. It has an **iterative solution too**, which turns out to be the same algorithm that trains neural networks in Lesson 9. And its coefficients are **readable**: a linear model tells you, in the units of the problem, what it thinks each feature is worth.

That last property is why linear models remain in production in medicine, credit scoring and public policy long after more accurate methods exist. When a decision has to be explained to the person

it affects, a model that says “your premium is higher because you live 8 km further out, and each kilometre is worth 6,500 euros” is worth more than an opaque one that is two points more accurate.

Everything in this lesson uses one dataset: 600 houses with six measurements and a price. It is synthetic, and that is deliberate — **we know the coefficients that generated the prices**, so every estimate can be checked against the truth. No real dataset lets you do that, and it is the only way to say honestly whether an estimate is good.

2. The model and the cost

2.1 What a linear model claims

For a house with features $x_1, \dots, x_n$ the model predicts

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = w^\top x + b$$

Read as a sentence, this says: **each feature contributes a fixed amount per unit, and the contributions add up**. Every extra square metre is worth the same 2,400 euros whether it is the fortieth or the two-hundredth. Every kilometre from the centre costs the same regardless of the neighbourhood.

That is a strong claim and it is often false. A hundred and first square metre probably is not worth what the fiftieth was; the value of a garage probably depends on whether there is street parking. The right response is not to abandon linear models but to know what you have assumed — Section 5 shows how to check, and how to relax the assumption when it fails.

2.2 Why squared error

The picture first. We need a single number saying how badly a candidate model is doing, so that “fitting” becomes “make this number small”. The obvious candidate — add up the errors — fails immediately, because a prediction 50,000 too high and one 50,000 too low would cancel to zero and look perfect.

So we need every error to count as a positive amount. Squaring does that, and it also decides something less obvious: how much worse one large mistake is than several small ones. Squaring says *much* worse.

Now the definition. The **mean squared error** is

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

Two questions are worth answering before accepting it.

Why squared, rather than absolute? Three reasons, in increasing order of importance. Squaring is differentiable everywhere, while $|x|$ has a corner at zero that complicates optimisation. Squaring penalises one large error more than several small ones — being wrong by 40,000 euros on one house costs the same as being wrong by 20,000 on four, which matches most intuitions about what a bad prediction is. And under the assumption that the noise is Gaussian, minimising squared error

is exactly maximum likelihood estimation, which is the deep reason and the one that makes the choice more than a convention.

The consequence is worth stating plainly: **squared error is sensitive to outliers**, because a single absurd value contributes its error squared. A house mispriced by a factor of ten will drag the whole fit towards itself. Lesson 2's outlier discussion is not decoration; it is a prerequisite for this lesson.

Where does the $\frac{1}{2}$ come from? Differentiating a square brings down a factor of 2, and the half cancels it, leaving a clean gradient. It has no effect on where the minimum is — scaling a function by a constant does not move its minimiser — so it is pure convenience.

2.3 A worked example

Take three houses and a candidate model $\hat{y} = 2400 \cdot \text{area} + 45000$. The error is $\hat{y} - y$ throughout, as in Section 2.2 — negative where the model has underpriced the house:

Area (m ²)	True price (€)	Predicted (€)	Error (€)	Error ²
80	240,000	237,000	−3,000	9.0×10^6
120	320,000	333,000	+13,000	1.69×10^8
200	540,000	525,000	−15,000	2.25×10^8

The cost is $\frac{1}{2 \times 3}(9.0 \times 10^6 + 1.69 \times 10^8 + 2.25 \times 10^8) \approx 6.7 \times 10^7$.

Notice how the arithmetic behaves. The 3,000 euro error contributes 2% of that total; the two large ones contribute the other 98%. Halving the small error would cut the cost by under 2% — you would barely see it move. Halving the 15,000 error would cut it by **42%**. **Squared error spends its attention on the worst predictions**, and that is a design decision you are making whether or not you notice it.

3. The exact solution

3.1 The normal equation, derived

The picture first. Each column of your data matrix is a direction you are allowed to move in. Any prediction the model can make is some combination of those directions, so the set of achievable predictions is a flat surface — a plane, in three dimensions — sitting inside the space of all possible answers.

The true prices are a point that almost certainly does **not** lie on that surface: no combination of area and age reproduces them exactly. So the best you can do is find the point on the surface closest to it, which is its **shadow** — the perpendicular projection.

Perpendicular is the whole content of the method. If the leftover error had any component lying *along* the surface, you could have moved in that direction and done better, so you were not at the closest point. At the optimum the error must be at right angles to every feature — which is exactly what the algebra below says, and why the equations are called *normal*.

Now the notation. Collect the training data into a matrix. Let X be $m \times (n + 1)$, with a column of ones in front so the intercept is just another coefficient, and let θ hold b followed by w . Then all m predictions at once are $X\theta$ and the cost is

$$J(\theta) = \frac{1}{2m} \|X\theta - y\|^2 = \frac{1}{2m} (X\theta - y)^\top (X\theta - y)$$

Expand the product:

$$J(\theta) = \frac{1}{2m} (\theta^\top X^\top X\theta - 2\theta^\top X^\top y + y^\top y)$$

using $\theta^\top X^\top y = y^\top X\theta$, both being scalars. Now differentiate with respect to θ . Two standard matrix identities do the work: $\nabla_\theta(\theta^\top A\theta) = 2A\theta$ for symmetric A , and $\nabla_\theta(\theta^\top c) = c$. So

$$\nabla_\theta J = \frac{1}{m} (X^\top X\theta - X^\top y)$$

Set it to zero and the $\frac{1}{m}$ drops out:

$$X^\top X\theta = X^\top y \quad \implies \quad \boxed{\theta = (X^\top X)^{-1} X^\top y}$$

These are the **normal equations**. The name comes from geometry: the residual vector $y - X\theta$ is normal — perpendicular — to the space spanned by the columns of X . The fit is the orthogonal projection of y onto that space, which is a satisfying way to see why least squares is natural rather than arbitrary.

Is this really a minimum? The second derivative is $\frac{1}{m} X^\top X$, which is positive semi-definite for any X — for any vector v , $v^\top X^\top X v = \|Xv\|^2 \geq 0$. So the cost is convex, and any stationary point is a global minimum. This is a genuinely useful property that most of the methods later in the course do not have: **there are no local minima to get stuck in.**

3.2 A worked example, by hand

Three houses, one feature. Area 80, 120, 200; price 240k, 320k, 540k (in thousands, to keep the numbers readable).

$$X = \begin{pmatrix} 1 & 80 \\ 1 & 120 \\ 1 & 200 \end{pmatrix}, \quad y = \begin{pmatrix} 240 \\ 320 \\ 540 \end{pmatrix}$$

$$X^\top X = \begin{pmatrix} 3 & 400 \\ 400 & 60800 \end{pmatrix}, \quad X^\top y = \begin{pmatrix} 1100 \\ 165600 \end{pmatrix}$$

Both are worth computing by hand once. The off-diagonal entry of $X^\top X$ is $80 + 120 + 200 = 400$ and the corner is $80^2 + 120^2 + 200^2 = 60,800$; the second entry of $X^\top y$ is

$$80 \cdot 240 + 120 \cdot 320 + 200 \cdot 540 = 19,200 + 38,400 + 108,000 = 165,600$$

The determinant of $X^\top X$ is $3 \times 60800 - 400^2 = 22400$, so

$$(X^\top X)^{-1} = \frac{1}{22400} \begin{pmatrix} 60800 & -400 \\ -400 & 3 \end{pmatrix}$$

and

$$\theta = \frac{1}{22400} \begin{pmatrix} 60800 \cdot 1100 - 400 \cdot 165600 \\ -400 \cdot 1100 + 3 \cdot 165600 \end{pmatrix} = \frac{1}{22400} \begin{pmatrix} 640000 \\ 56800 \end{pmatrix} \approx \begin{pmatrix} 28.57 \\ 2.536 \end{pmatrix}$$

So $b \approx 28,600$ euros and $w \approx 2,536$ euros per square metre. Three points, two parameters, and a slope within 6% of the 2,400 that generated the data — on three houses. Notebook 1 does the same thing on 450 and gets 2,410.

Check it the other way round, because a second route is the only real protection against an arithmetic slip. For one feature the least-squares slope is S_{xy}/S_{xx} with $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y})$ and $S_{xx} = \sum(x_i - \bar{x})^2$. With $\bar{x} = 133.\bar{3}$ and $\bar{y} = 366.\bar{6}$ that gives $18,933.3/7,466.7 = 2.536$, and $b = \bar{y} - w\bar{x} = 366.67 - 2.5357 \times 133.33 \approx 28.57$. The two routes agree, as they must — they are the same equations.

3.3 When the exact solution is not available

The formula requires inverting $X^\top X$, and that fails or becomes useless in three situations you will meet.

Redundant columns. If two features are exact linear combinations of each other — the same measurement in metres and in feet — then $X^\top X$ is singular and has no inverse. Notebook 3 constructs exactly this case.

Nearly redundant columns. Far more common, and worse, because nothing fails visibly. The inverse exists but is enormous, and small changes in the data produce large changes in θ . Notebook 3 shows a coefficient moving from 9,261 to 45,307 across four random splits of the same dataset.

Size. Inverting an $n \times n$ matrix costs on the order of n^3 operations. At a thousand features that is a billion; at a hundred thousand it is out of reach. Every large model in this course is trained iteratively, and the next section is why.

4. Gradient descent

4.1 The update rule, derived

The picture first. You are standing on a hillside in fog and want to reach the bottom. You cannot see the valley, but you can feel which way the ground slopes under your feet. So you take a step downhill, feel again, and repeat.

That is the whole algorithm. The gradient is the direction of steepest *ascent*, so we walk against it; the learning rate is how long a stride we take. The danger is equally intuitive: stride too far and you cross the valley and end up higher on the opposite slope.

Now the notation. When the exact solution is unavailable, start somewhere and walk downhill. The gradient points in the direction of steepest increase, so we step against it:

$$\theta \leftarrow \theta - \alpha \nabla J(\theta)$$

For squared error the components are worth deriving individually, because their form says something. Writing the error of example i as $e^{(i)} = \hat{y}^{(i)} - y^{(i)}$:

$$\frac{\partial J}{\partial w_j} = \frac{\partial}{\partial w_j} \left[\frac{1}{2m} \sum_i (e^{(i)})^2 \right] = \frac{1}{m} \sum_i e^{(i)} \frac{\partial e^{(i)}}{\partial w_j} = \frac{1}{m} \sum_i e^{(i)} x_j^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_i e^{(i)}$$

Read the first one. Each example pulls the coefficient in proportion to two things: how wrong the prediction was, and how large that feature was for that example. A 200 m² house that is badly mispriced moves the area coefficient much more than a 40 m² house with the same error. The gradient is a weighted vote, and the weights are the feature values — which is precisely why features on wildly different scales cause trouble, as Lesson 2 derived.

The intercept gradient has no x in it: every example gets an equal vote, since the intercept applies equally to all of them.

4.2 A worked step

Two houses: (80 m², 240k) and (200 m², 540k). Start at $w = 0$, $b = 0$, with $\alpha = 10^{-5}$.

Predictions are both 0, so the errors are -240 and -540 (in thousands).

$$\frac{\partial J}{\partial w} = \frac{1}{2} [(-240)(80) + (-540)(200)] = \frac{-19200 - 108000}{2} = -63600$$

$$\frac{\partial J}{\partial b} = \frac{-240 - 540}{2} = -390$$

Both gradients are negative, so both parameters increase:

$$w \leftarrow 0 - 10^{-5}(-63600) = 0.636, \quad b \leftarrow 0 - 10^{-5}(-390) = 0.0039$$

Look at the asymmetry. After one step the slope has moved 163 times further than the intercept, purely because areas are around 140 and the intercept’s “feature” is always 1. With a shared learning rate, the intercept will still be crawling towards its value long after the slope has arrived — which is exactly what notebook 1 shows after 4,000 iterations.

The cure is Lesson 2’s: scale the features, and both directions move at comparable speed.

4.3 Choosing the learning rate

The step size α is the one parameter gradient descent cannot choose for itself, and both failure modes are worth recognising on sight.

Too small and progress is slow but monotone: the cost falls at every iteration and simply takes too many of them.

Too large and the step overshoots the minimum and lands somewhere higher up the far side. The next step overshoots further. The cost *increases*, often explosively, and you see `nan` within a few dozen iterations.

Lesson 2 derived the threshold: for a quadratic bowl of curvature c , the update converges only if $\alpha < 2/c$, and with several features the binding constraint is the largest curvature — the largest feature variance. This is the same fact from the other side: **the safe learning rate is set by your worst-scaled feature**, which is a good reason to scale them all.

The practical recipe: start at 0.01 on scaled features, watch the cost, divide by three if it rises. Lesson 5 replaces the recipe with a search.

4.4 How stretched the valley is

The picture first. Section 4.3 said the safe learning rate is set by the largest curvature. That leaves an obvious question: what about the *smallest*?

A cost surface whose curvature is the same in every direction is a round bowl, and gradient descent walks straight to the bottom of it. When one direction is far steeper than another the bowl becomes a long, narrow ravine. You now have a problem, because **one step size has to serve every direction at once**. The steep direction sets the limit — step further and it diverges — so the shallow direction is stuck with a stride far too short for it, and crawls.

The ratio of the steepest curvature to the shallowest is the **condition number** of the design matrix. It is, near enough, what sets the number of iterations you will need.

On the housing data, computed with `numpy.linalg.cond` on the design matrix:

Design matrix	Condition number
The six features as recorded	285
The same six, standardised	3.4
Standardised, plus <code>area_sqft</code>	2,286

Read the first two rows together: **scaling is worth a factor of about 80 here**, and that factor is iterations you do not have to run. This is Lesson 2’s argument for scaling arriving from the optimisation side, and it is the same fact as the 163-to-1 asymmetry worked out in Section 4.2 — that asymmetry is what a condition number measures.

The third row is Section 3.3’s “nearly redundant columns” with a number attached. Adding a column that duplicates another, even with a rounding error between them, multiplies the condition number by roughly 670. That is why the coefficients in Section 7.3 are unusable while the predictions remain fine: the ravine is almost perfectly flat along the direction that trades `area_sqm` against `area_sqft`, so the fit has almost no basis for choosing a point along it — but every point along it predicts equally well.

The same number governs the exact solution, which is the tidy part: a large condition number is simultaneously why $(X^\top X)^{-1}$ is untrustworthy and why gradient descent is slow. One quantity, both failure modes.

5. Curves, and the price of flexibility

5.1 A linear model that bends

“Linear” refers to the coefficients, not the inputs. Nothing prevents us from handing the model x^2 as an additional column:

$$\hat{y} = w_1x + w_2x^2 + b$$

This is still least squares — the same normal equation, the same code — on a design matrix with an extra column. The model is linear in w , which is all the derivation in Section 3 ever required.

The same trick covers interactions (x_1x_2), logarithms, and any other transformation you can compute. It is the reason linear models remain useful on relationships that are not straight lines.

5.2 What it costs

Notebook 2 fits a curve — daily energy consumption against outdoor temperature, with a minimum around 18 °C — on 21 training observations. Here is the whole lesson in one table, measured with the root mean squared error (RMSE) — the square root of the mean squared error, which puts the number back into the units of the target:

Degree	Training RMSE	Test RMSE
1	113.3	212.1
2	17.6	17.4
3	17.5	16.9
6	17.0	23.3
9	16.5	118.2
12	16.3	182.0

Training error falls at every step. From 113 down to 16, monotonically, without ever suggesting that anything is going wrong. **Test error falls to degree 3 and then climbs** — by degree 12 it is ten times worse than the best.

Had we chosen the model by training error, we would have selected degree 12: the worst of the six, and the training error would have congratulated us the whole way. This is Lesson 1’s empirical-versus-expected risk argument with numbers in it, and it is why Section 2.3 of Lesson 1’s handout matters.

5.3 The mechanism

Why does a flexible model behave badly, rather than merely using its flexibility where it is needed?

Look at the coefficients. The degree-2 fit has a largest coefficient of 411. The degree-12 fit needs 247,514 — nearly a thousand times larger.

They are large because they work in **opposition**. With 21 points and 12 coefficients there are many ways to pass close to every point, and the arrangement least squares finds involves terms that nearly cancel: one pushing the curve up where the next pushes it down, the cancellation failing exactly where a training point sits. Between the points, nothing constrains the swing.

This observation is the whole basis of the next section. If large opposing coefficients are the symptom, **charge for coefficient size** and the symptom should disappear.

6. Regularisation

6.1 Two penalties

Add a price for size to the cost:

$$J_{\text{ridge}}(w) = \text{MSE}(w) + \lambda \sum_{j=1}^n w_j^2 \qquad J_{\text{lasso}}(w) = \text{MSE}(w) + \lambda \sum_{j=1}^n |w_j|$$

Ridge (also called L_2 or Tikhonov regularisation) charges the sum of squares; Lasso charges the sum of absolute values. The parameter $\alpha \geq 0$ sets the exchange rate between fitting the data and keeping coefficients small; at $\lambda = 0$ both reduce to ordinary least squares.

A note on names. The literature writes this penalty strength as λ , which is what we use, and reserves α for the learning rate of Section 4. scikit-learn, unhelpfully, calls the penalty parameter **alpha** — so `Ridge(alpha=0.01)` in code is $\lambda = 0.01$ in these pages.

The intercept is never penalised. It is not a claim about any feature, only where the surface sits, and shrinking it would bias every prediction towards zero. Both **Ridge** and **Lasso** in scikit-learn handle this for you.

Scaling is mandatory here, and for a reason that follows directly from the formula: the penalty is a sum over coefficients, so a feature measured in metres and the same feature measured in kilometres attract penalties differing by a factor of a thousand. Lesson 2 introduced scaling for the sake of optimisation speed; here it is a matter of correctness.

6.2 Ridge has a closed form

The picture first. Section 3.3 said least squares fails when two columns carry the same information: the matrix cannot be inverted, because there is a direction in which moving the coefficients changes nothing at all. The problem is flat in that direction, so there is no unique lowest point.

Ridge tilts the floor. By charging for coefficient size it makes every direction cost something, so the flat valley acquires a slope and a single lowest point appears. That is why Ridge always has an answer where least squares has none.

Now the algebra. Ridge, unlike Lasso, can be solved exactly. Differentiating

$$J(\theta) = \frac{1}{2m} \|X\theta - y\|^2 + \lambda \|\theta\|^2$$

gives, by the same steps as Section 3.1 plus $\nabla_{\theta}(\lambda\theta^{\top}\theta) = 2\lambda\theta$:

$$X^{\top}X\theta - X^{\top}y + 2m\lambda\theta = 0 \quad \implies \quad \boxed{\theta = (X^{\top}X + \tilde{\lambda}I)^{-1}X^{\top}y}$$

writing $\tilde{\lambda} = 2m\lambda$ for brevity. Compare it with the ordinary normal equation: the only change is $\tilde{\lambda}I$ added to the matrix being inverted.

Why there is a factor to keep track of at all. Our cost divides the fit term by $2m$ but not the penalty, so the two are measured on different scales and the $2m$ reappears in the solution. Write the cost as $\|X\theta - y\|^2 + \lambda\|\theta\|^2$ instead — no $1/2m$ — and the solution is exactly $(X^{\top}X + \lambda I)^{-1}X^{\top}y$, which is the form most textbooks print.

Neither convention is more correct, and the models they produce are identical once λ is rescaled. But it does mean **a penalty strength is only meaningful alongside the cost function it belongs to**: `Ridge(alpha=1)` in scikit-learn is not the same number as a λ of 1 taken from a paper, and comparing them directly is a mistake worth not making. This is the same trap as the naming clash in Section 6.1, one level deeper.

That small change is why Ridge is numerically well behaved. $X^{\top}X$ can be singular — Section 3.3 — but $X^{\top}X + \tilde{\lambda}I$ never is for $\tilde{\lambda} > 0$: adding a positive constant to the diagonal shifts every eigenvalue up by $\tilde{\lambda}$, so none of them can be zero. Ridge always has a unique solution, even when least squares has none or infinitely many.

6.3 Why Lasso reaches zero and Ridge does not

The picture first. Think of the penalty as a budget. You may spend a fixed total on coefficients, and you want the best fit that money can buy. The two methods differ only in how they charge you — by the square of each coefficient, or by its absolute value — and that changes the *shape* of what you can afford.

Draw the affordable region in two dimensions. Charging squares gives a circle; charging absolute values gives a diamond standing on its corners. The best fit you can afford sits where the region first touches the contours of the error.

A circle is smooth, so that contact happens at a generic point with both coefficients non-zero. A diamond has **corners, and its corners lie on the axes** — a corner is a point where one coefficient is exactly zero. Corners stick out, so they get touched first.

That is the entire reason Lasso produces zeros and Ridge does not. Now the same thing formally.

Rewrite each as a **constrained** problem, which is equivalent by Lagrange duality: minimise the squared error subject to a budget on the coefficients.

$$\text{Ridge: } \min_w \text{MSE}(w) \quad \text{s.t.} \quad \sum_j w_j^2 \leq t \qquad \text{Lasso: } \min_w \text{MSE}(w) \quad \text{s.t.} \quad \sum_j |w_j| \leq t$$

In two dimensions the ridge constraint region is a **disc**; the lasso region is a **diamond** with corners on the axes. The squared-error contours are ellipses centred on the least-squares solution, and the constrained optimum is where the smallest ellipse first touches the region.

A disc has no corners: the contact point is almost always in a generic position with both coordinates non-zero. A diamond has corners, and its corners sit **exactly on the axes** — a corner is where one coordinate is zero. An expanding ellipse is disproportionately likely to touch a corner first, because the corner protrudes.

So the sparsity of Lasso is not a numerical accident and not a tolerance threshold. It is what happens when a constraint region has corners on the axes.

6.4 Worked: what a penalty buys

Notebook 3 applies Ridge to the degree-12 disaster from Section 5:

Model	Training RMSE	Test RMSE	Largest $ w $
No penalty	16.3	182.0	247,514
Ridge, $\lambda = 0.01$	18.0	22.8	365
Ridge, $\lambda = 1$	44.5	105.2	156
Ridge, $\lambda = 100$	96.0	227.6	6

A penalty of $\lambda = 0.01$ — barely a touch — cuts the largest coefficient from 247,514 to 365 and the test error from 182 to 23, which is the noise floor. An eightfold improvement, from one number.

And notice the training column: it gets worse at every step. That is the trade being made explicitly — we accept a worse fit on the data we have, in exchange for a better fit on data we do not. The exchange stops paying: at $\lambda = 100$ the test error is 228, worse than no penalty at all, because the model has become too rigid to follow the curve.

The useful range here spans four orders of magnitude, which is the practical argument for why λ cannot be guessed and must be searched. Lesson 5 provides the machinery.

6.5 Worked: Lasso as a feature selector

On the housing data, raising λ drops features in a specific order:

	Features kept
1,000	all six
10,000	five — garage dropped
20,000	three — area , bedrooms , bathrooms
40,000	one — area_sqm

The order is not arbitrary. **garage** goes first: on a common scale it has the smallest effect of the six. **area_sqm** survives longest because it explains the most price per unit of coefficient spent.

Nobody told Lasso which features mattered. It was told to keep the total size of its coefficients within a budget, and this is the arrangement that buys the most accuracy for that budget. **The selection depends entirely on λ** , which is a choice you must justify — a fact often forgotten when Lasso output is presented as an objective ranking of importance.

7. Reading coefficients

7.1 What a coefficient means, exactly

A coefficient w_j is the change in the prediction for a one-unit change in x_j , **with every other feature held fixed**.

That last clause is doing enormous work and is almost always glossed over. It means the coefficient on **bedrooms** is not “what a bedroom is worth” — it is what a bedroom is worth *to a house whose floor area does not change*, which is a strange object: a house that gains a bedroom without gaining any space.

7.2 Worked: when the estimate can be trusted

Notebook 1 fits all six features and compares against the truth:

Feature	True	Estimated	Error	Correlation with area
area_sqm	2,400	2,421	+0.9%	—
age_years	−1,800	−1,791	−0.5%	−0.02
distance_km	−6,500	−6,455	−0.7%	0.00
garage	12,000	10,850	−9.6%	−0.04
bathrooms	14,000	15,229	+8.8%	0.49
bedrooms	9,000	7,503	−16.6%	0.77

Read the first and last columns together. The three features **uncorrelated** with area are recovered to within 1%. The two correlated with it are out by 9% and 17%, and the worst estimate belongs to the most correlated feature.

A coefficient is only as trustworthy as the independence of its feature. With 450 rows and 18,000 euros of noise, this is the precision the data supports — and on a real dataset the same wobble would be there with nothing to reveal it.

7.3 Worked: when it cannot be trusted at all

Notebook 3 adds **area_sqft**, the same measurement in different units (correlation 0.999999). Ordinary least squares on four different random splits of the same data:

Split	area_sqm	area_sqft
0	9,261	−640
1	14,535	−1,127
2	45,307	−3,989
3	39,061	−3,402

The coefficient on area varies by a factor of five, and the coefficient on its duplicate swings to compensate. Predictions are fine throughout — their combined effect is stable at about 2,420 — but the explanation is noise.

With Ridge at $\lambda = 10$, the same four splits give roughly 38,000–41,000 on both columns, splitting the effect evenly. Two moderate coefficients cost less under a squared penalty than one huge positive and one huge negative.

So Ridge is not only a cure for overfitting. It is what makes coefficients readable when features overlap — which in real data they nearly always do.

7.4 The warning that belongs on every coefficient

A coefficient describes an association in the training data. It is not a causal effect, and the model has no way to tell the difference.

If houses near the centre are older *and* more expensive, “age” and “distance” will divide the effect between them in whatever proportion best fits the sample. Neither number tells you what would happen if you moved a house.

The professional habit: **state what a coefficient is conditional on**, and be suspicious of any interpretation phrased as an intervention.

8. Before the next lesson

1. Work the three notebooks in order. Notebook 1 is the one to do slowly.
2. Read Sections 3.1, 4.1 and 6.3 with a pen — those three derivations are examinable.
3. Take the quiz in [Quizzes/](#).
4. **Complete the homework** in `Exercises/03_regression.md`, due at the start of Lesson 4.

Lesson 4 keeps the same machinery and changes the target from a number to a category, which turns out to require a different cost function — and the reason why is one of the derivations you will be asked for.

Further reading

Resource	Type	Why read it
scikit-learn: linear models	Official docs	The full family, with the mathematics stated compactly
Hastie, Tibshirani & Friedman, <i>Elements of Statistical Learning</i> , ch. 3	Book	The definitive treatment of linear regression and shrinkage; §3.4 is Ridge and Lasso
Tibshirani, <i>Regression Shrinkage and Selection via the Lasso</i> (1996)	Paper	The original, and unusually readable
Hoerl & Kennard, <i>Ridge Regression</i> (1970)	Paper	Written to solve the collinearity problem of Section 7.3, not the overfitting one
Gelman & Hill, <i>Data Analysis Using Regression</i> , ch. 3–4	Book	On interpreting coefficients honestly, which is Section 7 done properly